Practical Problem Set 6 and Set 7 solutions - 0726

ABHILASHA DAS

2026-03-17

```r
rm(list=ls())
library(ISLR)

## Warning: package 'ISLR' was built under R version 4.3.3

library(leaps)

## Warning: package 'leaps' was built under R version 4.3.3

attach(Hitters)
View(Hitters)
#cleaning the data
data1=na.omit(Hitters)
#Split the data randomly into train-test keeping 80% as training sample and
remaining as test sample
set.seed(123)
id=1:263
train_index = sample(id,210)
train_data = data1[train_index, ]
test_data = data1[-train_index, ]
# Perform the best subset selection on the training sample. While choosing
the best model fit up to a 19 variable model. Choose the best model according
to adjusted R squared, BIC , AIC and Cp values. In case the best model
differs according to the selection criteria, report the best fitted model
according to each selection criterion

reg1=regsubsets(Salary ~ ., data = train_data,nvmax=19)
reg_summary=summary(reg1);reg_summary

## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train_data, nvmax = 19)
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
```

```
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: exhaustive
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    " "   " "    "*"
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    " "
"*"
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    " "
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 7  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 8  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 9  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 10  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 11  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 12  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 13  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    " "   " "    "*"
"*"
## 14  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    " "   " "    "*"
"*"
## 15  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 16  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 17  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
```

```
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 5  ( 1 )  " "    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 7  ( 1 )  " "    " "     "*"       "*"     "*"     " "    " "
## 8  ( 1 )  " "    " "     "*"       "*"     "*"     "*"    " "
## 9  ( 1 )  " "    " "     "*"       "*"     "*"     "*"    "*"
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

```r
adjr2=which.max(reg_summary$adjr2)
cp=which.min(reg_summary$cp)
bic=which.min(reg_summary$bic)
cat("Adjusted R2:", adjr2, "nCp:", cp, "BIC:", bic)
```

```
## Adjusted R2: 12 nCp: 11 BIC: 5
```

```r
# Find the training MSE and test MSE for each of the best models selected
according to different criterion as mentioned in 3 and compare their values

predict.regsubsets=function(object, newdata, id) {
  form=as.formula(object$call[[2]])
  mat=model.matrix(form, newdata)
  coefi=coef(object, id = id)
  xvars=names(coefi)
  mat[, xvars] %*% coefi
}
metrics=list(AdjR2=adjr2, Cp=cp, BIC=bic)

for (name in names(metrics)) {
  id=metrics[[name]]
 train_pred=predict.regsubsets(reg1, train_data, id)
 train_mse=mean((train_data$Salary - train_pred)^2)
  test_pred=predict.regsubsets(reg1, test_data, id)
  test_mse=mean((test_data$Salary - test_pred)^2)

  cat(name, "Model (Size", id, ") - Train MSE:", train_mse, "Test MSE:",
```

```
test_mse, "\n")
}
```

```
## AdjR2 Model (Size 12 ) - Train MSE: 88935.8 Test MSE: 130066.9
## Cp Model (Size 11 ) - Train MSE: 89790.63 Test MSE: 125652.3
## BIC Model (Size 5 ) - Train MSE: 97026.43 Test MSE: 153090.5
```

```
#  Repeat 3-4 using forward stepwise selection method
regfit_fwd=regsubsets(Salary ~ ., data =
train_data,nvmax=19,method="forward")
fwd_summary=summary(regfit_fwd);fwd_summary
```

```
## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train_data, nvmax = 19,
##     method = "forward")
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: forward
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    " "   " "    "*"
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
"*"
```

```
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 7  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 8  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 9  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 10  ( 1 ) " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 11  ( 1 ) " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 12  ( 1 ) " "   " "  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 13  ( 1 ) "*"   " "  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 14  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 15  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 16  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 17  ( 1 ) "*"   "*"  " "   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  "*"    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 5  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 7  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 8  ( 1 )  "*"    " "     "*"       "*"     " "     " "    " "
## 9  ( 1 )  "*"    " "     "*"       "*"     " "     " "    "*"
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

```r
# Repeat 3-4 using backward stepwise selection method
regfit_bwd=regsubsets(Salary ~ ., data =
train_data,nvmax=19,method="backward")
bwd_summary=summary(regfit_bwd);bwd_summary
```

```
## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train_data, nvmax = 19,
##     method = "backward")
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: backward
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    "*"   " "    " "
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    "*"   " "    " "
" "
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    " "
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 7  ( 1 )  " "   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 8  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 9  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
```

```
" "
## 10  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 11  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 12  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 13  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 14  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
" "
## 15  ( 1 ) "*"   "*"  " "   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
" "
## 16  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
" "
## 17  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 5  ( 1 )  " "    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 7  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 8  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 9  ( 1 )  " "    " "     "*"       "*"     "*"     " "    " "
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"

# 1. Split the data set into a training set and a test set randomly keeping
100 observations in test set and remaining in the training set.
library(ISLR)
hitters_clean=na.omit(Hitters)
set.seed(123)
test_indices=sample(1:nrow(hitters_clean), 100)
test_data=hitters_clean[test_indices, ]
train_data=hitters_clean[-test_indices, ]
```

```r
# 2. Train the model using least squares (linear regression)
lm_fit=lm(Salary ~ ., data = train_data)
train_pred=predict(lm_fit, train_data)
train_mse=mean((train_data$Salary-train_pred)^2)

# 3. Fit ridge regression models on the training set, with grid values of λ.
# Report the following:
# (i) the dimension of the coefficient matrix so obtained.
# (ii) The L2 norm of the regression coefficients corresponding to the 50th
and 60th choice of λ. Comment on the output.
# (iv) Test MSE corresponding to these two choices of λ.
test_pred=predict(lm_fit, test_data)
test_mse=mean((test_data$Salary-test_pred)^2)
cat("Training MSE:", round(train_mse, 2), "\n")
```

```
## Training MSE: 105334.2
```

```r
cat("Test MSE:    ", round(test_mse, 2), "\n")
```

```
## Test MSE:     87803.91
```

```r
library(glmnet)
```

```
## Warning: package 'glmnet' was built under R version 4.3.3

## Loading required package: Matrix

## Warning: package 'Matrix' was built under R version 4.3.3

## Loaded glmnet 4.1-8
```

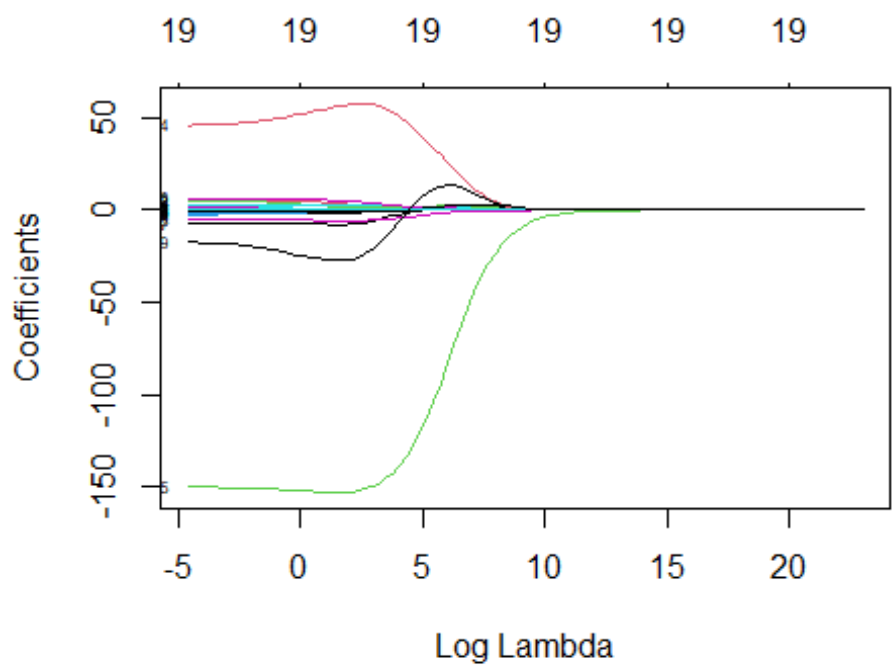
```r
x_train=model.matrix(Salary ~ ., train_data)[, -1] # Remove intercept column
y_train=train_data$Salary
x_test=model.matrix(Salary ~ ., test_data)[, -1]
y_test=test_data$Salary
# Define the grid for lambda
grid=10^seq(10, -2, length = 100)
# Fit Ridge Regression (alpha = 0)
ridge_mod=glmnet(x_train, y_train, alpha=0, lambda=grid, standardize=TRUE)
dim(coef(ridge_mod))
```

```
## [1]  20 100
```

```r
# Extract coefficients (excluding intercept) for 50th and 60th lambda
coef_50=predict(ridge_mod, type="coefficients", s=grid[50])[-1]
coef_60=predict(ridge_mod, type="coefficients", s=grid[60])[-1]
# Calculate L2 Norms: sqrt(sum(beta^2))
norm_50=sqrt(sum(coef_50^2))
norm_60=sqrt(sum(coef_60^2))
cat("L2 Norm (50th lambda):", norm_50, "\n")
```

```
## L2 Norm (50th lambda): 7.076105
```

```r
cat("L2 Norm (60th lambda):", norm_60, "\n")

## L2 Norm (60th lambda): 66.36125

pred_50=predict(ridge_mod, s = grid[50], newx = x_test)
mse_50=mean((pred_50 - y_test)^2)

pred_60=predict(ridge_mod, s = grid[60], newx = x_test)
mse_60=mean((pred_60 - y_test)^2)
cat("Test MSE (50th lambda):", mse_50, "\n")

## Test MSE (50th lambda): 167303.1

cat("Test MSE (60th lambda):", mse_60, "\n")

## Test MSE (60th lambda): 97935.56

# 4. Make a line plot (for all the predictors) of the standardized
coefficients against the grid value of λ chosen by you. Interpret the graph.
plot(ridge_mod, xvar = "lambda", label = TRUE)
```



Ans4. The plot above displays the coefficient paths for the Ridge Regression model as a function of the tuning parameter λ. Based on the visualization, we can draw the following conclusions:

Shrinkage Toward Zero: As $\log(\lambda)$ increases (moving from left to right), the L_2 penalty becomes more severe, forcing the regression coefficients to shrink toward zero.

Bias-Variance Tradeoff: On the far left (low λ), the coefficients are close to the Least Squares estimates, which typically have low bias but high variance. As we move right, we introduce bias by constraining the coefficients, which helps reduce the model's variance and potentially lowers the Test MSE.

5. How would the graph in (4) change if instead of λ you plot on the x axis the ratio of l2 norm of the ridge regression coefficient to the least square regression coefficient.

Ans5. If the x-axis is replaced by the ratio

$$\frac{|\ |\beta_{ridge}|\ |_2}{|\ |\beta_{OLS}|\ |_2}$$

then the horizontal axis represents the relative size of ridge coefficients compared to the OLS coefficients. In this case: The ratio equals 1 when λ = 0 because ridge coefficients are identical to OLS estimates. As λ increases, the ridge coefficients shrink, so the ratio decreases toward 0. The shape of the coefficient paths would remain the same, but the x-axis interpretation changes: Instead of showing λ directly, it now shows the degree of shrinkage. Moving left to right would correspond to increasing shrinkage of coefficients. Thus, the graph would still show coefficients shrinking toward zero, but the scale would represent how much smaller the ridge coefficients are compared to the OLS solution rather than the value of λ.

```
# 6. Run the Ridge regression on training data by choosing λ by cross-
validation. Find the test MSE for the optimal choice of λ obtained using
cross validation.
set.seed(123)
cv_ridge=cv.glmnet(x_train, y_train, alpha = 0)
best_lambda_ridge=cv_ridge$lambda.min
ridge_pred_opt=predict(cv_ridge, s=best_lambda_ridge, newx=x_test)
mse_ridge=mean((ridge_pred_opt-y_test)^2)
cat("Optimal Ridge Lambda:", best_lambda_ridge, "\n")
cat("Ridge Test MSE:      ", mse_ridge, "\n")

# 7. Fit a LASSO model on the training set, with λ chosen by cross-
validation.Report the test MSE, along with the number of non-zero coefficient
estimates.
cv_lasso=cv.glmnet(x_train,y_train,alpha=1)
best_lambda_lasso=cv_lasso$lambda.min
```

```
# Test MSE for LASSO
lasso_pred=predict(cv_lasso, s=best_lambda_lasso, newx=x_test)
mse_lasso=mean((lasso_pred - y_test)^2)
# Number of non-zero coefficients
lasso_coef=predict(cv_lasso, type="coefficients", s=best_lambda_lasso)
non_zero_lasso=sum(lasso_coef!= 0)-1
cat("LASSO Test MSE:          ", mse_lasso, "\n")
cat("Non-zero LASSO Predictors:", non_zero_lasso, "\n")

# 8. Fit an Elastic Net model on the training set, with λ1 and λ2 chosen by
cross-validation. Report the test MSE. Also report the predictors with non-
zero coefficients.
cv_en=cv.glmnet(x_train, y_train, alpha = 0.5)
best_lambda_en=cv_en$lambda.min
en_pred=predict(cv_en,s=best_lambda_en,newx=x_test)
mse_en=mean((en_pred-y_test)^2)
en_coef=predict(cv_en,type="coefficients",s=best_lambda_en)
en_active_predictors=row.names(en_coef)[which(en_coef!=0)]

cat("Elastic Net Test MSE:", mse_en, "\n")
cat("Active Predictors:   ", paste(en_active_predictors[-1], collapse=", "),
"\n")
```